Teaching a simulated human arm to reach

A reinforcement-learning model: its design, the constraints on the arm, the reward function, and how it was trained

CP493 Directed Research — Wilfrid Laurier University

Ranjot Sandhu

September 2026

SAC	5.9 h	92%	95%
algorithm Soft Actor-Critic	to train one model (1 M steps, CPU)	success on unseen random targets	on freely clicked point sets

The model is a two-joint, muscle-driven model of a human right arm, built to human proportions and limits. A neural network controls its four muscles; it was trained, by trial and reward alone, to reach any three points in its workspace and hold the last one still. No demonstrations or motion data were used.

1. Summary

What was built. A physics simulation of a human right arm seen from the side: upper arm and forearm-plus-hand, driven by four Hill-type muscles (an agonist and an antagonist at the shoulder and the elbow), with gravity, realistic segment masses, human range of motion and human torque limits. A neural-network policy reads 18 numbers describing the arm and its target, and outputs the four muscle activations 50 times a second.

How it learns. With **Soft Actor-Critic** (SAC), an off-policy deep reinforcement-learning algorithm. The policy is never shown how to move; it is only scored, every 20 ms, by a reward function (section 6) that pays for approaching and holding the target and charges for effort, jerky or overly fast motion, and straining the joints.

What it can do. The deployed model (stage **R3**) reaches three randomly placed targets and holds the last one for half a second in **92 %** of fresh episodes (mean of 5 independently trained seeds), and in **95 %** of sets of three points clicked freely by a user. It never exceeds a hard physical limit, and it stiffens its arm the way people do, by co-contracting opposing muscles while it holds still.

How long it takes. One model — 1,000,000 simulated steps — trains in about **5.9 hours** on a CPU (section 2). The whole project ran 63 training runs: 36.7 million steps and 193 seed-hours of compute.

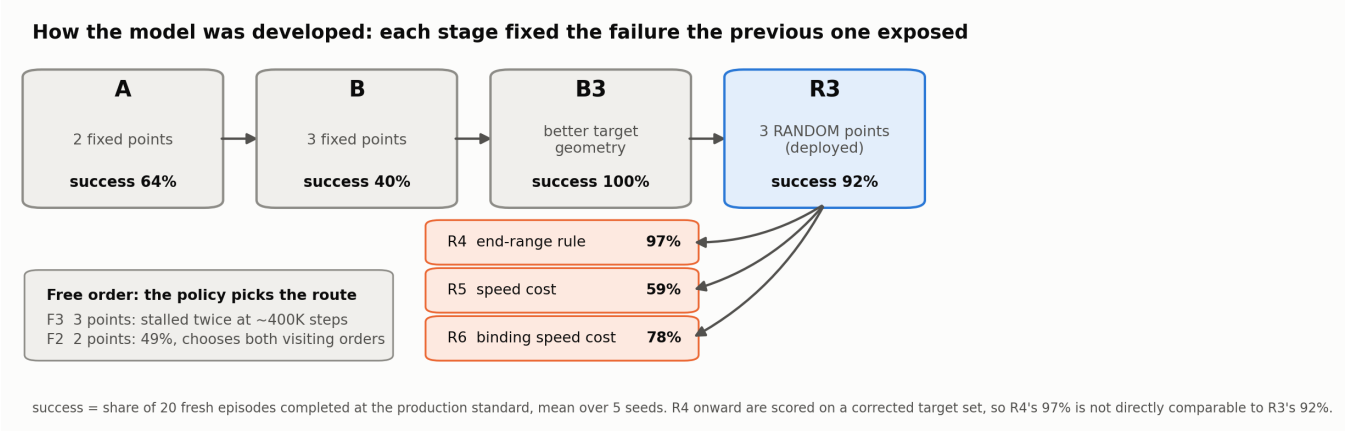


Figure 1. The development path. Each stage fixed the failure the previous one exposed; the random-point stage R3 is the one that generalises to targets it has never seen.

2. How long it takes to train one model

5.9 h	1,000,000	2,000	47
wall-clock time, one model	simulated steps (20 ms each)	practice episodes of 10 seconds	steps per second on a CPU

One model takes about 5.9 hours to train. That is the measured wall-clock time of the deployed R3 model: 1,000,000 environment steps at 47 steps per second. A step is 20 ms of simulated time, so the arm practises for 20,000 seconds — about **5.6 hours of simulated movement**, split into 2,000 attempts of 10 seconds each. Training therefore ran at roughly real-time speed.

Hardware and set-up. Training ran on the CPU; for networks this small the GPU was measured 3.5–5× *slower*, because moving tiny batches to and from the GPU costs more than the arithmetic it saves. Five models (five random seeds) were trained side by side, four CPU threads each, so a whole five-seed stage also finishes in about 5–6 hours of wall-clock time — R3 used 29 seed-hours in total. Each environment step triggers one gradient update of three 256×256 networks on a batch of 256 samples, and those updates, not the physics, dominate the time.

stage	what it trained	steps per model	hours per model	steps / s	seed-hours, whole stage
A	2 fixed targets	300 K	1.8	46	9
B	3 fixed targets (v1)	600 K	3.8	44	19
B3	3 fixed targets (v2)	600 K	3.0	55	15
R3	3 random targets (deployed)	1.0 M	5.9	47	29
R4	+ end-range rule	1.0 M	5.5	50	28
R5	+ speed cost	1.0 M	4.7	59	24
R6	+ binding speed cost	1.0 M	5.2	53	26
F2	2 targets, free order	500 K	2.3	61	11

Hours per model are wall-clock, measured with five seeds sharing the machine. Seed-hours sum all five.

One caveat. Every timing here was measured with five seeds sharing the machine. A single model trained alone, with the whole CPU to itself, was never timed; it would probably finish somewhat sooner than 5.9 hours, but by how much was not measured.

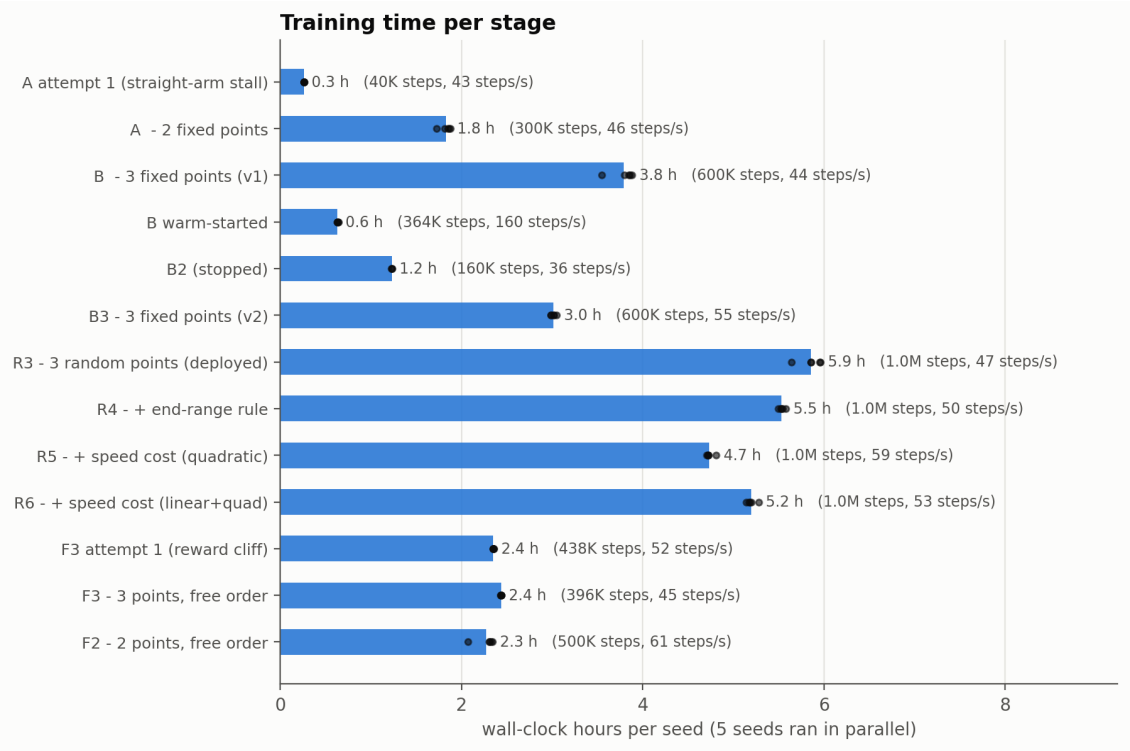


Figure 2. Wall-clock hours per model for every stage, including the failed attempts (dots are individual seeds).

2.1 The whole project

Counting every attempt, including the ones that failed, the project trained 63 models for 36.7 million steps in 193 seed-hours. Because each stage's seeds ran at the same time, that took about **40 hours** of machine time.

stage	what it was	seeds	steps per seed	hours per seed	seed-hours
A, attempt 1	stalled: straight arm on its elbow stop	5	40 K	0.3	1.3
A	2 fixed targets	5	300 K	1.8	9.1
B	3 fixed targets (v1)	5	600 K	3.8	18.9
B, warm-started from A	worse than training from scratch	5	364 K	0.6	3.2
B2	+ activation dynamics, acceleration cost; stopped	3	160 K	1.2	3.7
B3	3 fixed targets (v2)	5	600 K	3.0	15.1
R3	3 random targets (deployed)	5	1.0 M	5.9	29.3
R4	+ end-range rule	5	1.0 M	5.5	27.7
R5	+ speed cost	5	1.0 M	4.7	23.7
R6	+ binding speed cost	5	1.0 M	5.2	26.0
F3, attempt 1	stopped: reward cliff	5	438 K	2.4	11.8
F3	stopped: no progress	5	396 K	2.4	12.2
F2	2 targets, free order	5	500 K	2.3	11.4
total	**63 training runs**		**36.7 M**		**193**

3. The system

Every 20 ms the policy receives an **observation** of 18 numbers and chooses an **action**: four muscle activations between 0 and 1. The muscles turn activation into force according to their length and speed; the forces become joint torques; the rigid-body equations move the arm under gravity; the task checks whether the hand has reached or is holding its target; and the reward function scores what happened. The transition — what the policy saw, what it did, what it earned and what it saw next — is stored in a replay buffer, from which SAC samples batches to improve the policy.

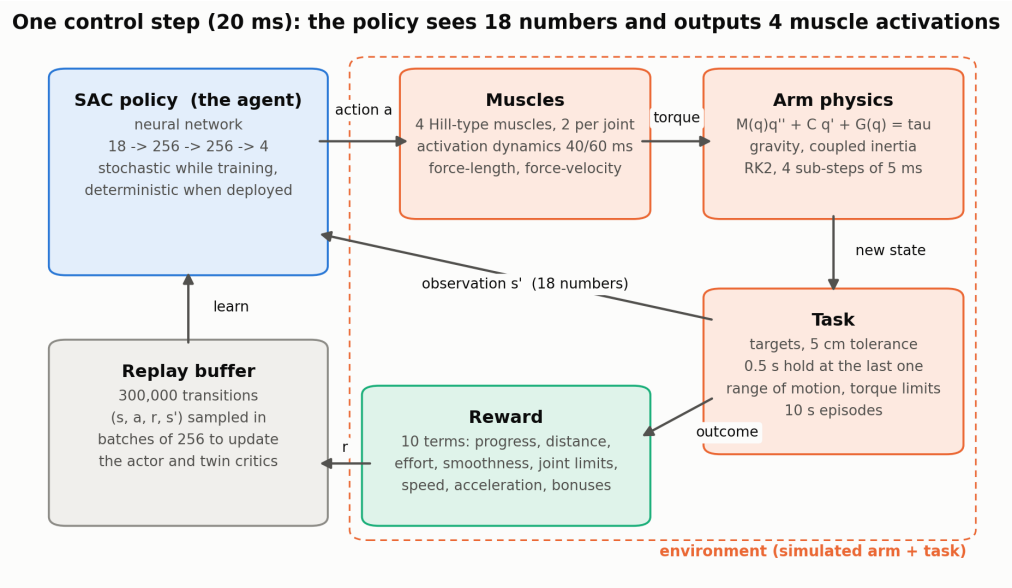


Figure 3. One control step. Everything inside the dashed boundary is the environment: the simulated arm, its muscles, the task and the reward. The agent only sees the observation and only acts through the four muscles.

3.1 What the policy sees and does

Observation — 18 numbers, all scaled to small ranges so the network trains well. Because the target is part of the observation, the same policy can reach any target; including the muscles' own activation state lets it account for their 40–60 ms lag.

#	observation component	count	scale / range
1–4	sine and cosine of the two joint angles	4	−1 … 1
5–6	joint angular velocities	2	÷ 8 rad/s
7–8	vector from the hand to the active target	2	÷ reach 0.65 m
9	distance from the hand to the active target	1	÷ reach
10–11	hand velocity	2	÷ 2 m/s
12	which target is active (progress through the list)	1	0 … 1
13	hand currently inside the goal (the reached test)	1	0 or 1
14	hold progress at the current target	1	0 … 1
15–18	current activation of each of the four muscles	4	0 … 1

Action — 4 numbers in [0, 1]: the commanded activation of the shoulder's two muscles and the elbow's two muscles. The policy never sets a torque or a position directly.

4. The arm and every constraint on it

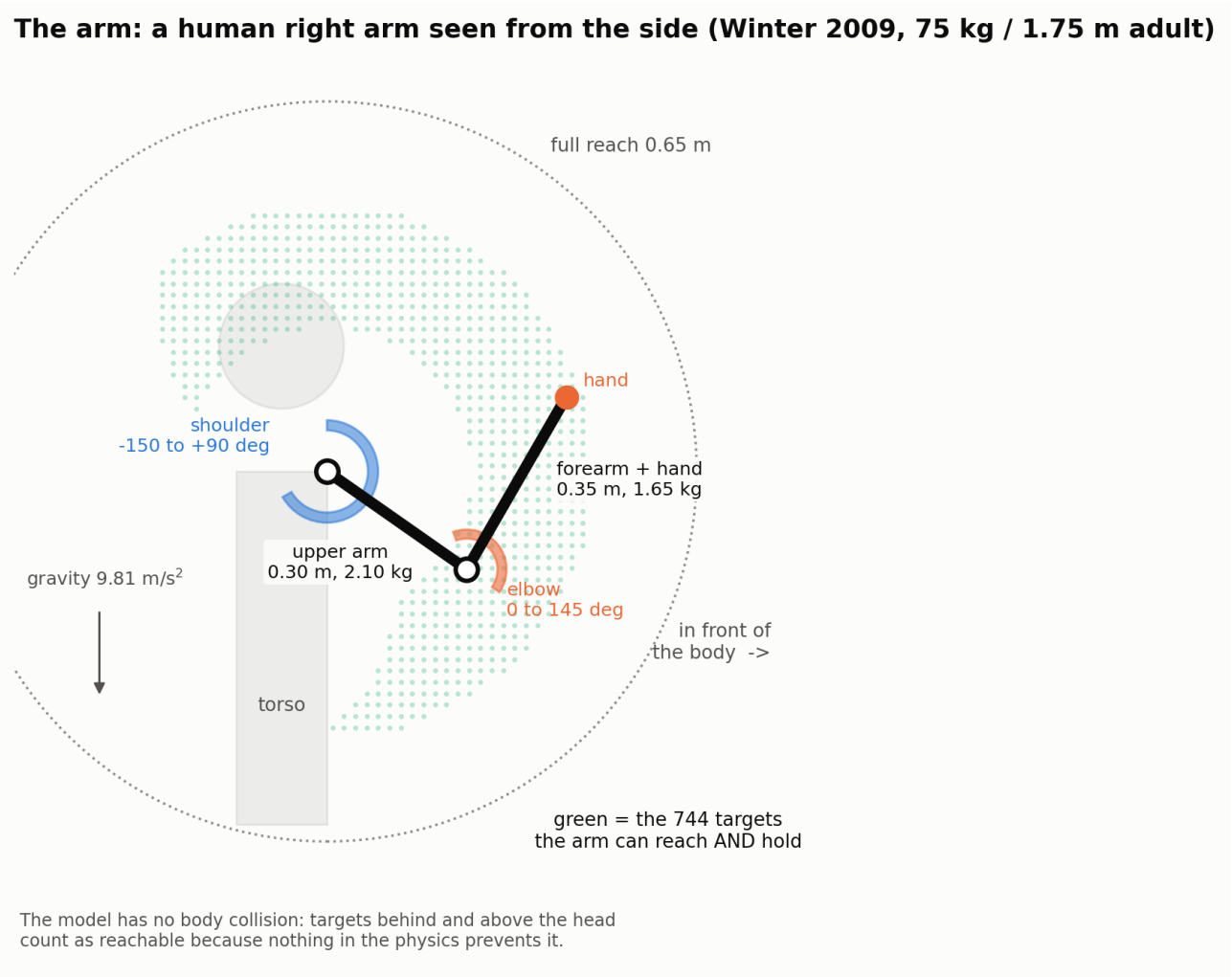


Figure 4. The arm at a representative pose, its joint ranges, and — in green — every target it is allowed to be asked to reach: points it can both reach and hold still.

4.1 Body dimensions

Segment proportions follow Winter (2009) for a 75 kg, 1.75 m adult.

segment	length	mass	centre of mass from joint	moment of inertia (about COM)
upper arm	0.30 m	2.10 kg	0.131 m	0.0196 kg·m²
forearm + hand	0.35 m	1.65 kg	0.175 m	0.0237 kg·m²

4.2 Constraints at a glance

Hard limits are properties of the simulated body: nothing the policy does can exceed them, and none was exceeded over every step of 30 audited episodes. **Soft** limits are costs in the reward: the policy can exceed them, and pays for it. **Task** rules define success.

constraint	value	kind	how it is enforced
shoulder range of motion	−150° … +90° (flexion −60° … 180°)	**hard**	the integrator clamps the angle and removes velocity into the stop
elbow range of motion	0° … 145°	**hard**	as above
joint torque	shoulder 25 N·m, elbow 12 N·m	**hard**	net torque clipped; about 40 % of an adult's maximum
joint speed	8 rad/s	**hard**	safety clamp; muscles weaken with speed well before
muscle activation	0 … 1 per muscle	**hard**	the bounds of the action space
activation rise / fall	40 ms / 60 ms	**hard**	first-order filter: muscle force cannot change instantly
gravity	9.81 m/s²	body	gravity vector in the equations of motion
end-range stiffness	20 N·m/rad inside 10° of each stop	body	a spring that pushes the joint back out, like soft tissue
joint damping	0.30 N·m·s/rad	body	viscous resistance
muscle force–length, force–velocity	Hill-type (Thelen 2003; Hill 1938)	body	muscle force depends on fibre length and speed (4.3)
hand speed	cap 1.5 m/s (R3)	soft	a reward cost above the cap
hand acceleration	reference 10 m/s²	soft	a reward cost (minimum-jerk proxy)
end-range posture	within 10° of a stop	soft	a reward cost for entering the zone
reached	within 5 cm, hand < 0.10 m/s, joints < 1.0 rad/s	task	all three at once
held	25 steps in a row (0.5 s)	task	counter +1 inside, −5 per step outside
episode length	500 steps × 20 ms = 10 s	task	time limit, not a failure

4.3 Physics

The arm obeys the rigid-body equations of motion for a two-link chain, with the full coupling between the joints and gravity acting on both segments:

$$M(q)\,\ddot{q} \;+\; C(q,\dot{q})\,\dot{q} \;+\; G(q) \;=\; \tau_{\text{muscle}} \;+\; \tau_{\text{passive}}$$

$$G(q) = g\,[(m_1 l_{c1} + m_2 L_1)\cos\theta_0 + m_2 l_{c2}\cos(\theta_0+\theta_1),\;\; m_2 l_{c2}\cos(\theta_0+\theta_1)]^T$$

$$\tau_{\text{passive}} = -B\,\dot{q} \;-\; K\max(0,\; q-(q_{\max}-\delta)) \;+\; K\max(0,\; (q_{\min}+\delta)-q)$$

Here q = (θ_0, θ_1) are the shoulder and elbow angles, M the 2×2 mass matrix, C the Coriolis and centrifugal terms, B = 0.30 N·m·s/rad the damping, K = 20 N·m/rad the end-range spring stiffness and δ = 10° its margin. The equations are integrated with a second-order (midpoint) method in four 5 ms sub-steps per 20 ms control step; total energy of the undamped arm drifts by only 0.3 % over 2 s, a check that the simulation is sound.

4.4 Muscles

Each joint has two lumped muscles pulling in opposite directions (shoulder: anterior vs posterior deltoid; elbow: biceps and brachialis vs triceps). A muscle's force is its activation times its maximum force, scaled by how its length and speed affect it:

$$F \;=\; a\cdot F_{\max}\cdot f_L(\ell)\cdot f_V(v), \qquad \tau_{\text{muscle}} = r\,(F_+ - F_-)$$

$$f_L(\ell) = \exp\!\left(-\left(\frac{\ell/\ell_{\text{opt}}-1}{0.45}\right)^2\right) \qquad (\text{force}-\text{length},\ \text{Thelen 2003})$$

$$f_V(v) = \frac{1-v}{1+v/0.25}\ \text{ for } 0\le v<1 \qquad (\text{force}-\text{velocity},\ \text{Hill 1938};\ \text{eccentric branch up to 1.5})$$

$$\ell = \ell_{\text{opt}} - s\,r\,(\theta-\theta_{\text{ref}}), \qquad v = \frac{s\,r\,\dot{\theta}}{\ell_{\text{opt}}\,V_{\max}}, \qquad V_{\max} = 6\ \ell_{\text{opt}}/\text{s}$$

$$\dot{a} = \frac{u-a}{\tau_{\text{act}}}, \qquad \tau_{\text{act}} = 40\text{ ms rising},\ \ 60\text{ ms falling} \qquad (\text{activation dynamics},\ \text{Zajac 1989})$$

joint	moment arm r	max force F_{max}	optimal length ℓ_{opt}	force reaches zero at
shoulder	0.04 m	625 N	0.20 m	30 rad/s
elbow	0.03 m	400 N	0.10 m	20 rad/s

Consequences worth noting: a muscle is weakest at the ends of the range (f_L never falls below 0.45 inside it) and weakens as it shortens faster; equal activation of both muscles gives no net torque but stiffens the joint — which the trained policy uses to hold still. There is deliberately no penalty on co-contraction.

4.5 Which targets the arm can be asked to reach

Random targets are drawn from a 2 cm grid, keeping only points that pass five rules. The result is the green region of Figure 4 — 744 points. When a user clicks a point that fails a rule, the interface moves it to the nearest point that passes.

rule	a target is allowed only if …	why
1 reachable	an elbow-down pose reaches it inside the range of motion	otherwise impossible
2 holdable	holding it needs ≤ 80 % of each joint's torque limit	otherwise the arm sags under gravity
3 not degenerate	it is at least 0.18 m inside the full-reach circle	near full reach a straight arm resting on its elbow stop counted as 'in goal'
4 distinct	targets are > 0.30 m apart, and away from the start pose	so a motionless arm can never satisfy two at once
5 comfortable	its holding pose clears every joint's end-range zone	otherwise the reward charges for the only pose that holds it

Rules 3 and 5 were added after failures: rule 3 when Stage B's arm learned to rest a straight arm on its elbow stop and be scored as holding, and rule 5 when targets needing an end-range pose proved measurably harder (75 % against 88 % success) for a reason the reward itself created.

5. The reinforcement-learning model

5.1 The algorithm: Soft Actor-Critic

SAC (Haarnoja et al. 2018) is an off-policy actor-critic method for continuous actions. Instead of maximising reward alone, it maximises reward *plus the entropy of the policy*, which keeps the policy exploring until the reward justifies committing to one way of moving:

$$J(\pi) = \sum_t \mathbb{E}[\, r(s_t, a_t) + \alpha \mathcal{H}(\pi(\cdot \mid s_t)) \,]$$

It suits this problem for three reasons: the action space is continuous (four activations); exploration matters, because the arm must discover strategies such as co-contracting to hold still; and being off-policy it reuses every past transition from a replay buffer, which makes it sample-efficient. SAC is also what Fischer et al. (2021) used to control a full biomechanical model of the upper extremity. The implementation here is Stable-Baselines3 2.7.1 (Raffin et al. 2021).

5.2 The networks

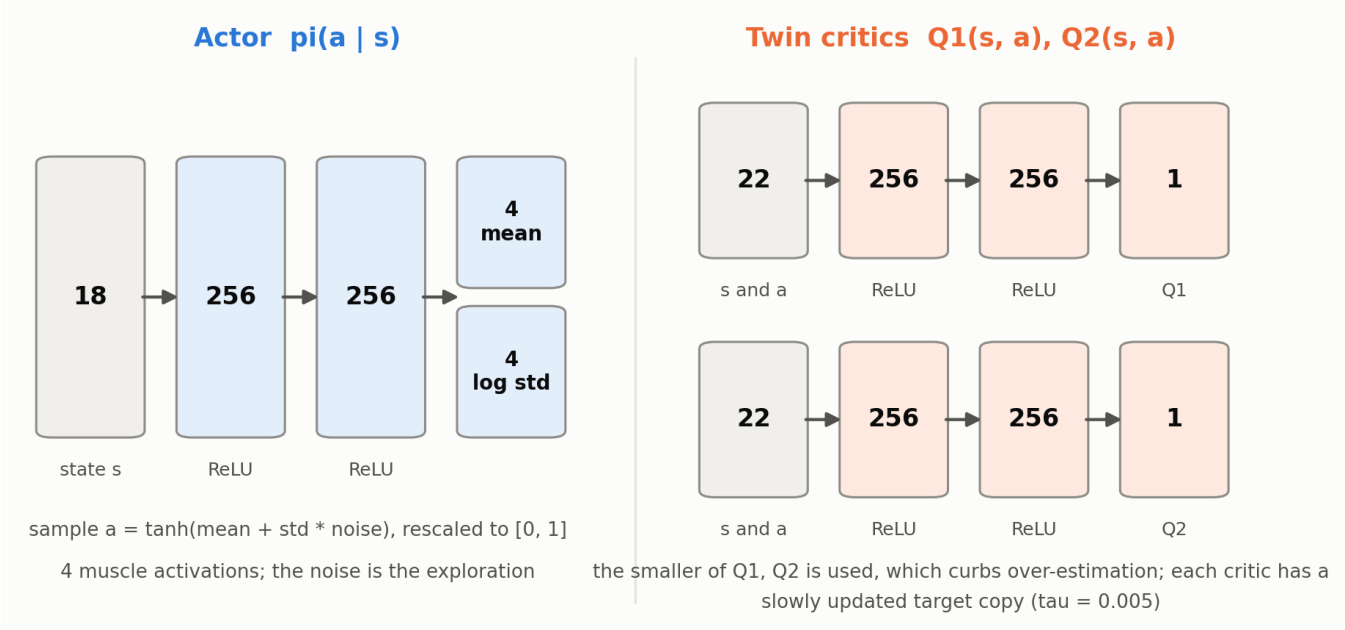


Figure 5. The actor maps the 18-number state to the mean and spread of a Gaussian over the 4 activations. Two critics each estimate the value Q of a state-action pair; SAC uses the smaller of the two to avoid over-estimating.

5.3 What is optimised

Each gradient step samples a batch of 256 transitions (s, a, r, s') from the replay buffer and updates four things:

$$y = r + \gamma(1-d)\Big(\min_{i=1,2} Q_{\bar{\theta}_i}(s', \tilde{a}') - \alpha \log \pi_\phi(\tilde{a}' \mid s')\Big), \qquad \tilde{a}' \sim \pi_\phi(\cdot \mid s')$$

$$\mathcal{L}_Q = \tfrac{1}{2} \sum_{i=1,2} \mathbb{E}\big[(Q_{\theta_i}(s,a) - y)^2\big] \qquad \text{(critics)}$$

$$\mathcal{L}_\pi = \mathbb{E}_s\Big[\, \alpha \log \pi_\phi(\tilde{a} \mid s) - \min_i Q_{\theta_i}(s, \tilde{a}) \,\Big], \qquad \tilde{a} = \tanh(\mu_\phi(s) + \sigma_\phi(s)\,\varepsilon) \qquad \text{(actor)}$$

$$\mathcal{L}_\alpha = -\mathbb{E}\big[\log\alpha\,(\log\pi_\phi(\tilde{a}\,|\,s)+\bar{\mathcal{H}})\big], \qquad \bar{\mathcal{H}}=-4 \qquad \text{(temperature)}$$

$$\bar{\theta}_i \leftarrow \tau\theta_i+(1-\tau)\,\bar{\theta}_i, \qquad \tau=0.005 \qquad \text{(target networks)}$$

d marks a true termination. Training episodes never terminate on success — they are cut off at the 10 s time limit — so the critic keeps valuing the future, and the policy learns to go on holding the target rather than to stop once it arrives. The temperature α is tuned automatically toward a target entropy of −4 (minus the number of actions).

5.4 Hyperparameters

hyperparameter	value	hyperparameter	value
networks	MLP 256 × 256, ReLU	discount γ	0.99 (≈ 2 s horizon)
critics	twin Q + target copies	learning rate	3×10^{-4} (Adam)
replay buffer	300,000 transitions	batch size	256
random warm-up	5,000 steps	target update τ	0.005
entropy α	automatic, target −4	updates per step	1
training length	1,000,000 steps	device	CPU
seeds	5 per stage (1–5)	episode	500 steps = 10 s

5.5 The curriculum

Asking for 5 cm precision and a half-second hold from the first step would leave the policy with almost no successes to learn from. Training therefore starts with a 15 cm tolerance and a short hold, and tightens only when the policy has earned it:

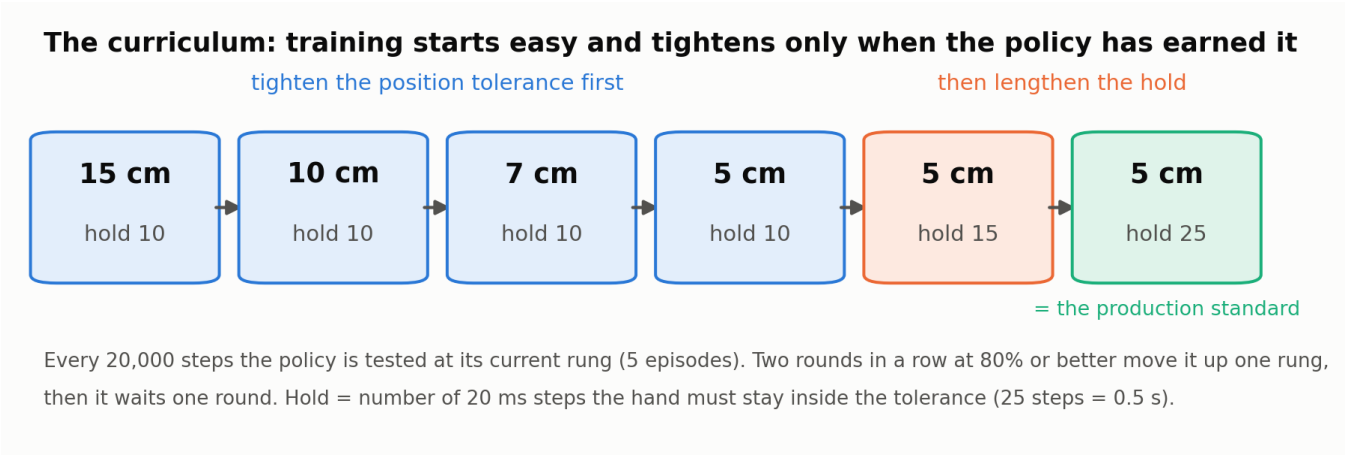


Figure 6. The six rungs of the curriculum.

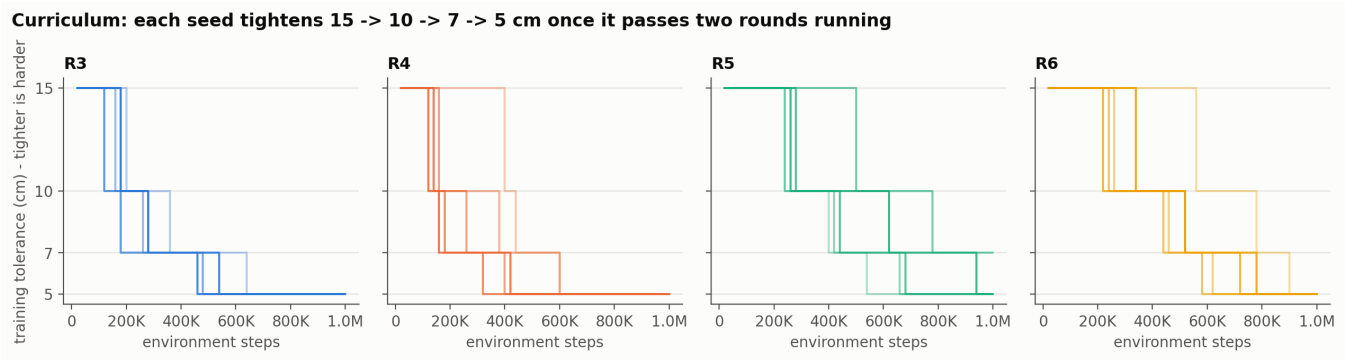


Figure 7. When each seed tightened its tolerance. Every seed of every random-point stage reached the final 5 cm rung.

Choosing the model to keep. Separately from the curriculum, every 20,000 steps the policy is also scored at the full production standard (5 cm, 0.5 s hold) on 10 fresh episodes. The best-scoring checkpoint is saved as the model; ties are broken by waypoints reached, then closeness, then speed.

6. The reward function

The reward is computed after every 20 ms step. It is the sum of ten terms. Distances are measured as fractions of the arm's reach R = 0.65 m. Let d be the distance from the hand to the active target, and define the **potential**

$$\Phi(s) \;=\; -\,\frac{d}{R}$$

so that Φ is higher (closer to zero) the nearer the hand is. The full reward is:

$$\begin{aligned}
r_t \;=\;& \quad W_{\text{shape}}\,(\gamma\,\Phi(s_{t+1}) - \Phi(s_t)) && \text{shaping}\\
&-\; W_{\text{dist}}\;d_{t+1}/R && \text{distance}\\
&-\; W_{\text{effort}}\;\|a_t\|^2 && \text{effort}\\
&-\; W_{\text{smooth}}\;\|\dot{q}_{t+1}\|\,/\,\omega_{\max} && \text{smoothness}\\
&-\; W_{\text{limit}}\;(n_{\text{stop}} + \sum_j e_j) && \text{joint limits}\\
&-\; (W_{\text{lin}}\,\delta + W_{\text{speed}}\,\min(\delta^2,\; C_{\text{speed}})), \quad \delta = \max(0,\; \|v\| - v_{\text{cap}}) && \text{hand speed}\\
&-\; W_{\text{accel}}\;\min((\|\dot{v}\|/A_{\text{ref}})^2,\; C_{\text{accel}}) && \text{hand acceleration}\\
&+\; R_{\text{goal}}\;\mathbb{1}[\text{in goal at the final target}] && \text{hold stream}\\
&+\; R_{\text{inter}}\;\mathbb{1}[\text{an intermediate target completed}] && \text{intermediate bonus}\\
&+\; R_{\text{succ}}\;\mathbb{1}[\text{the final hold completed,\;\; first time}] && \text{success bonus}
\end{aligned}$$

where a is the action, q̇ the joint velocities, ω_{max} = 8 rad/s, n_{stop} the number of joints on a hard stop, $e_j \in [0, 1]$ how far joint j has entered its 10° end-range zone, v the hand velocity and v̇ its acceleration. γ = 0.99 is SAC's discount.

6.1 What each term is for

term	weight in R3	what it does	why it is there
shaping	2	rewards moving toward the target, charges moving away	a learning signal from the first update; potential-based (Ng et al. 1999), so it cannot change which behaviour is optimal
distance	0.5	a steady cost for being far from the target	keeps a pull toward the goal even when the hand is still
effort	0.02	prefers smaller activations	a metabolic-cost proxy, kept small
smoothness	0.05	discourages needless joint speed	mild smoothing
joint limits	0.1	charges for resting on a stop or in end range	the first attempt rested a straight arm on its elbow stop
speed	cap 1.5 m/s, quadratic 0.5	charges for hand speed above the cap	without it the hand whipped at 4–5 m/s
acceleration	0.02	charges for sharp changes in hand velocity	produces the bell-shaped speed profile of human reaching
hold stream	+1 per step	pays for every step spent holding the final target	pays for holding, not touching; nothing is paid at intermediate targets
intermediate	+10	once per intermediate target completed	makes finishing a target worth leaving it
success	+50	once, when the final hold completes	marks the task done

6.2 The reward the deployed model learned from

Substituting the R3 weights (A_{ref} = 10 m/s², C_{accel} = 4; the linear speed term did not yet exist):

$$r_t = 2\,(0.99\,\Phi_{t+1} - \Phi_t) \;-\; 0.5\,\frac{d}{0.65} \;-\; 0.02\,\|a\|^2 \;-\; 0.05\,\frac{\|\dot{q}\|}{8} \;-\; 0.1\,(n_{\text{stop}} + \sum_j e_j)$$

$$-\; 0.5\,\max(0,\; \|v\| - 1.5)^2 \;-\; 0.02\,\min((\|\dot{v}\|/10)^2,\; 4) \;+\; 1\cdot\mathbb{1}[\ldots] \;+\; 10\cdot\mathbb{1}[\ldots] \;+\; 50\cdot\mathbb{1}[\ldots]$$

There is deliberately **no penalty on co-contraction**, because squeezing both muscles is how a muscle-driven arm stiffens to hold still — and the trained model does exactly that (co-contraction 0.48 while holding, against 0.23 while moving).

6.3 How the weights changed from stage to stage

The reward was refined as each stage exposed a problem. The weights below are read from each trained model's own saved record, not from the current code. 'off' means the term did not exist when that model was trained.

stage	shape	dist	effort	smooth	limit	v cap	speed quad.	speed lin.	quad. cap	accel	hold	inter.	success
A	2	0.5	0.02	0.05	0.1	off	off	off	off	off	1	10	50
B	2	0.5	0.02	0.05	0.1	1.5	0.5	off	∞	off	1	10	50
B3	2	0.5	0.02	0.05	0.1	1.5	0.5	off	∞	0.02	1	10	50
R3	2	0.5	0.02	0.05	0.1	1.5	0.5	off	∞	0.02	1	10	50
R4	2	0.5	0.02	0.05	0.1	1.5	0.5	off	∞	0.02	1	10	50
R5	2	0.5	0.02	0.05	0.1	1.2	10	off	2.25	0.02	1	10	50
R6	2	0.5	0.02	0.05	0.1	1.2	5	3	2.25	0.02	1	10	50
F2	2	0.5	0.02	0.05	0.1	1.5	0.5	off	∞	0.02	1	10	50

Stage A had no speed or acceleration cost; B added the quadratic speed cost; the acceleration cost arrived with B2 (a short attempt that was stopped) and stayed from B3 on; R5 lowered the speed cap to 1.2 m/s, raised the speed weight twenty-fold and capped the quadratic at 2.25; R6 added a linear speed term, because a purely quadratic cost has zero slope at the cap and so can never hold the hand's peak speed exactly at it.

6.4 Free-order variant

In the free-order experiments (F2, F3) the policy sees all targets at once and picks the order itself. The potential then becomes the length of the shortest route from the hand through every target not yet visited, U, which rewards choosing an efficient order:

$$\Phi(s) \;=\; -\,\frac{1}{R}\,\min_{\sigma\,\in\,\mathrm{perm}(U)}\left(\;\|p-g_{\sigma_1}\|\;+\;\sum_{i}\|g_{\sigma_i}-g_{\sigma_{i+1}}\|\;\right)$$

6.5 One episode, term by term

The deployed model on the three demonstration targets finishes in 99 steps (2.0 s) and earns +67.9. The bonuses for completing targets and holding the last one dominate; distance is the largest cost. A policy that does nothing for the whole 10 s scores about −355.

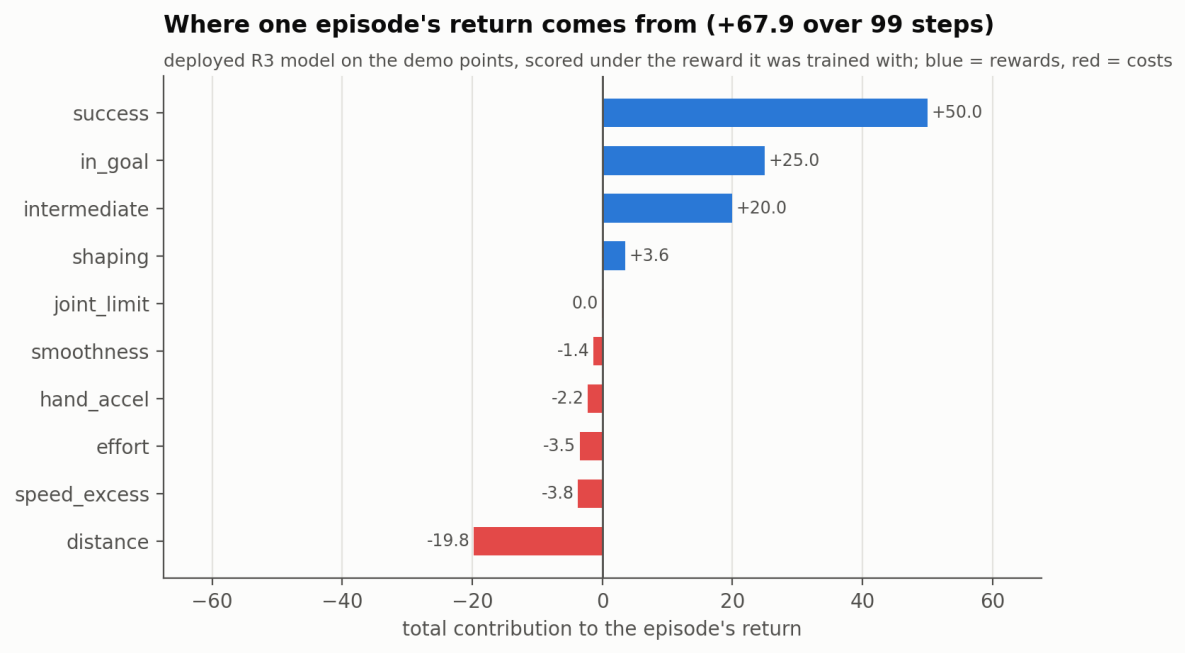


Figure 8. Each reward term's total over one episode of the deployed model.

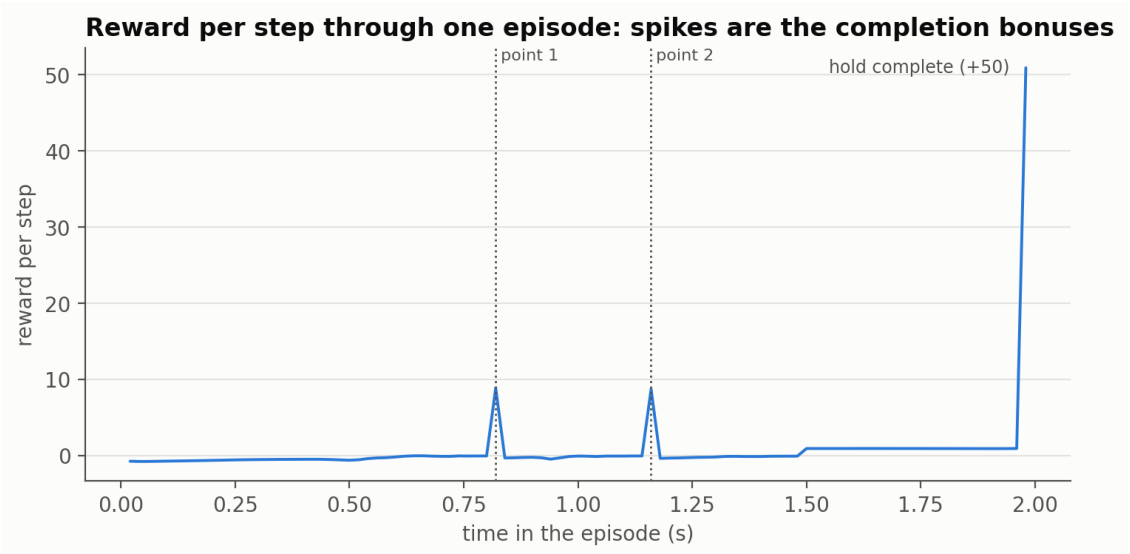


Figure 9. Reward per step through the same episode: +10 at each intermediate target, +50 when the final hold completes, and +1 per step while holding.

7. Training results

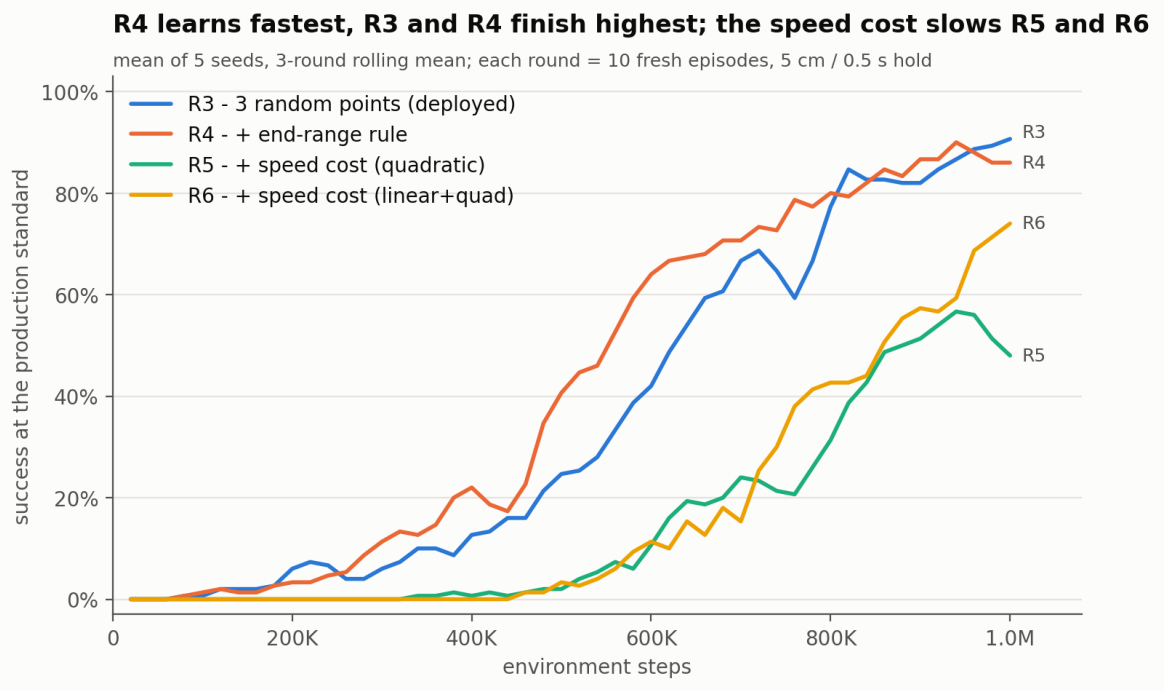


Figure 10. How success grew during training for the four random-target stages.

stage	task	seeds ≥ 80 %	success (mean ± sd over seeds)
A	2 fixed targets	3 / 5	64% ± 50%
B	3 fixed targets (v1)	2 / 5	40% ± 55%
B3	3 fixed targets (v2)	5 / 5	100% ± 0%
R3	3 random targets (deployed)	5 / 5	92% ± 6%
R4	+ end-range rule*	5 / 5	97% ± 3%
R5	+ speed cost (quadratic)	3 / 5	59% ± 41%
R6	+ binding speed cost	3 / 5	78% ± 24%
F2	2 random targets, free order	0 / 5	49% ± 12%

Final evaluation: each seed's best model on 20 fresh episodes at 5 cm and a 0.5 s hold. *R4 onward are scored on a corrected target set; on identical episodes R3 and R4 are statistically indistinguishable (94.4 % vs 93.0 %, p = 0.67).

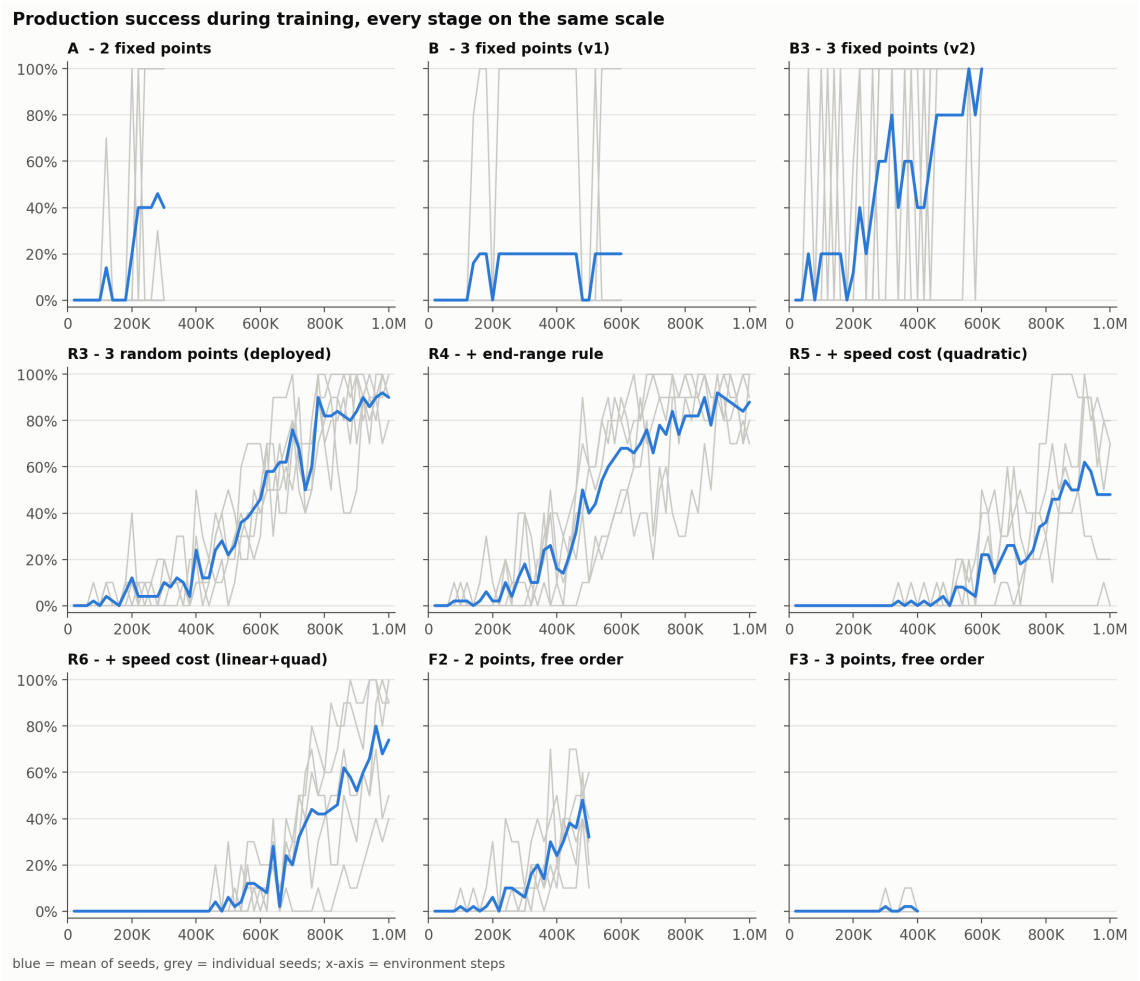


Figure 11. Learning curves for every stage on the same scale. Blue is the mean of the five seeds; grey lines are individual seeds.

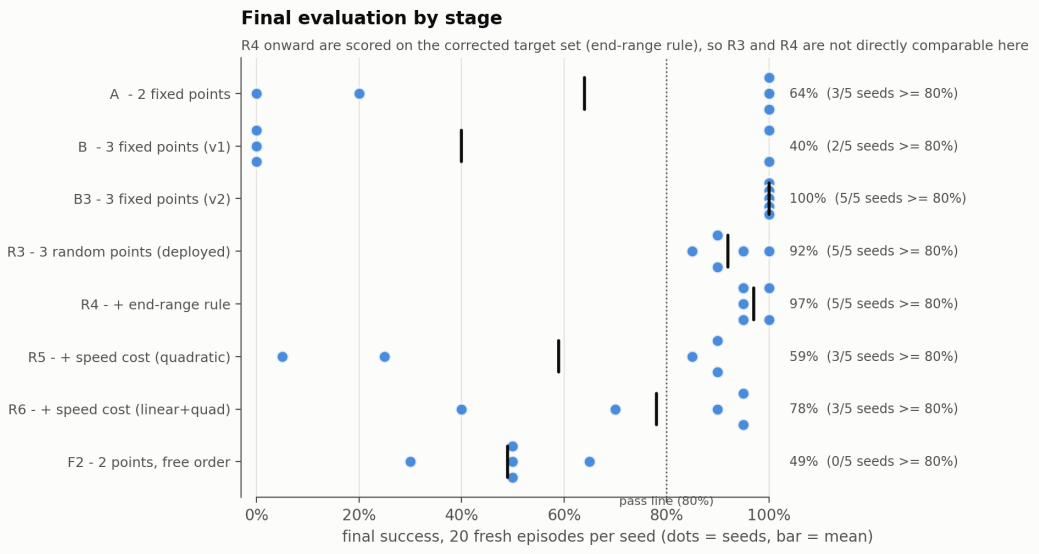


Figure 12. Final success of every seed of every stage.

7.1 A model that generalises, not one that memorises

A model trained on fixed targets learns one trajectory, not how to reach: B3, perfect on its own three targets, fails on any others. Trained on targets re-drawn every episode, R3 has to use the target in its observation, and generalises. The best seed of each, on the same 20 random target sets:

model	success	targets reached	final error
B3 (trained on 3 fixed targets)	0 %	0.25 of 3	24.7 cm
R3 (trained on random targets)	**100 %**	**3.00 of 3**	**2.1 cm**

7.2 Does it move like a person?

Six measures from the motor-control literature, compared with their human ranges. R3 is accurate but reaches fast (about 2 m/s peak against a human 1.0–1.5 m/s) with slightly curved paths. Adding a speed cost (R5, R6) brought the speed into the human range, but at a cost in reliability — a genuine trade-off, shown in Figure 14.

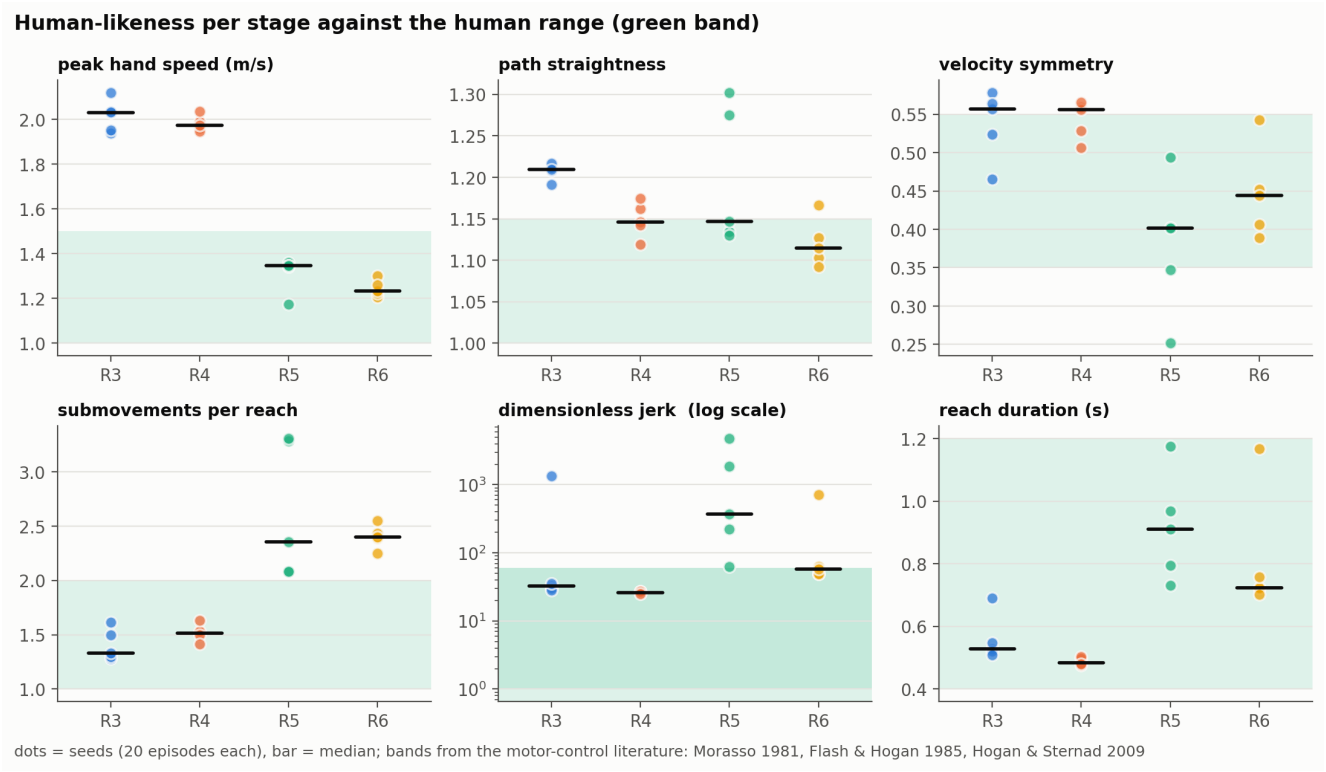


Figure 13. Human-likeness per stage. Dots are seeds, the bar is the median, the green band the human range.

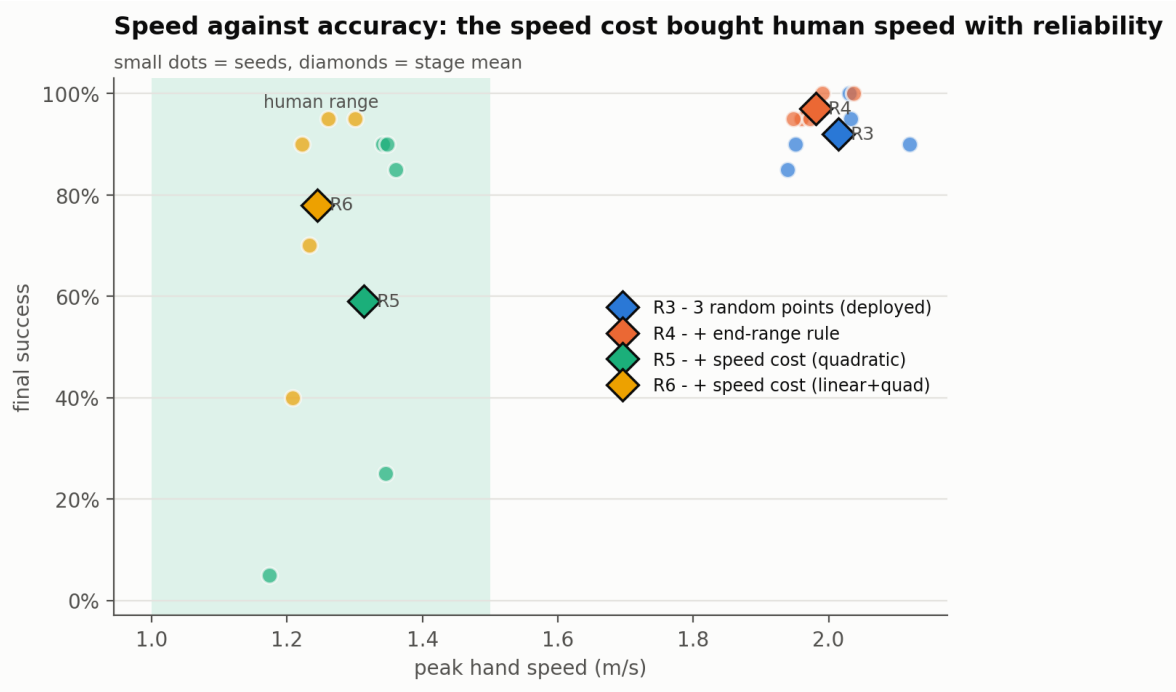


Figure 14. Every attempt to slow the hand to human speed also lowered success.

best model	peak hand speed	human measures in range	reaches all 3 clicked points
R3 (deployed)	2.14 m/s	4 of 6	**38 / 40 = 95 %**
R5	1.35 m/s	6 of 6	28 / 40 = 70 %
R6	1.30 m/s	5 of 6	17 / 40 = 42 %

These are each stage's best seed. Across all five seeds, by median, R6 is inside 5 of the 6 human ranges, R4 and R5 inside 4 and R3 inside 3. R3 is deployed because it is the most reliable.

7.3 Inside the learning algorithm

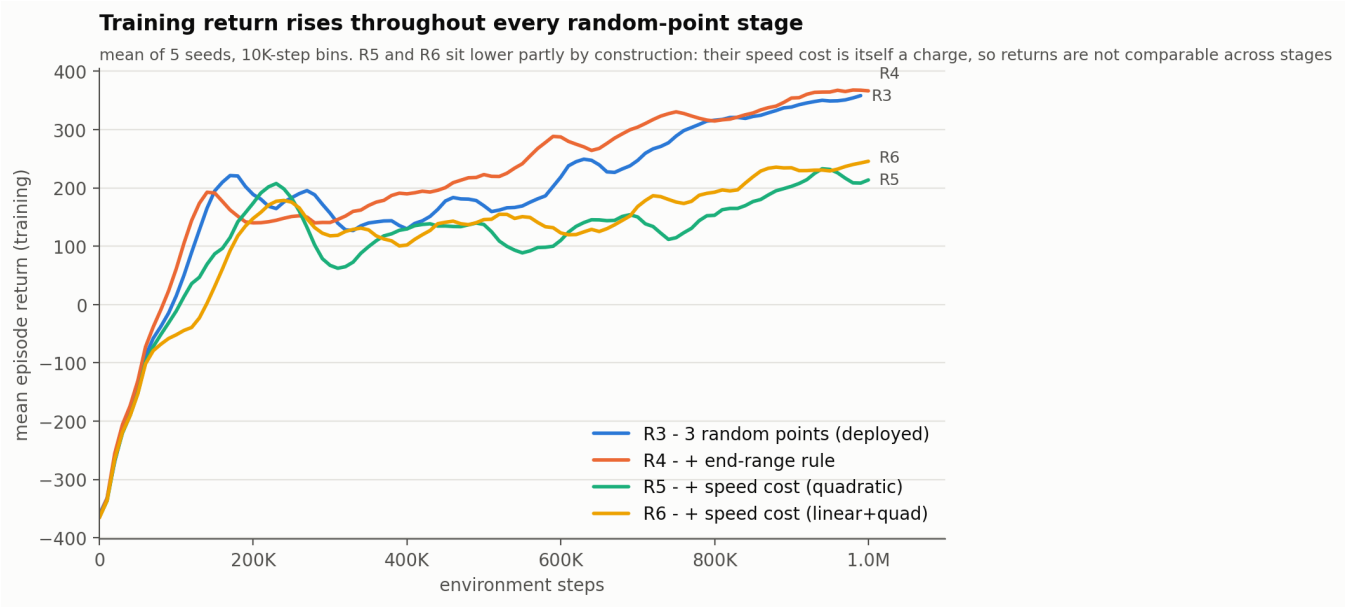


Figure 15. Return per training episode rises throughout. R5 and R6 sit lower partly because their speed cost is itself a charge.

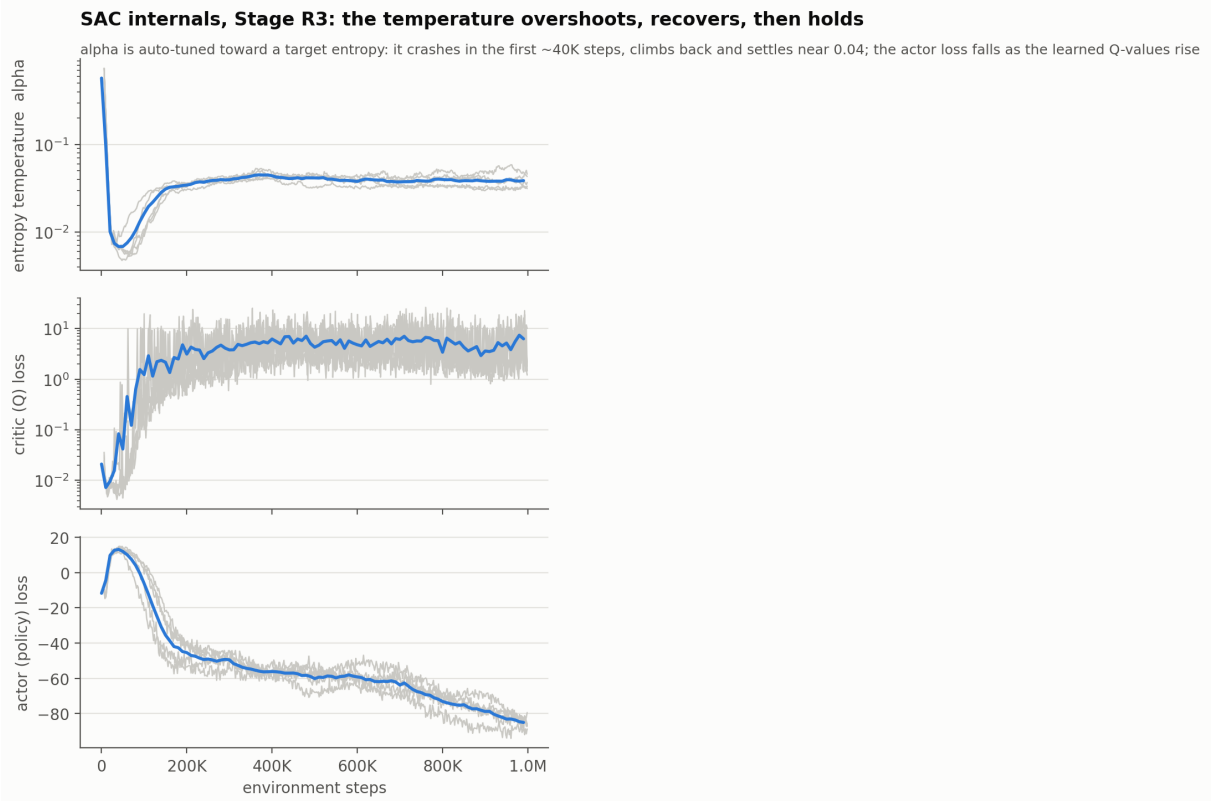


Figure 16. SAC's internal quantities during R3's training. The entropy temperature α crashes in the first 40,000 steps, recovers and settles near 0.04; the actor loss keeps falling as the learned values rise.

8. What did not work

Negative results are part of the record: each shaped the final design.

problem	what happened and what fixed it
A straight arm resting on its elbow stop	Stage A's first attempt stalled on every seed. A flexed resting pose, an end-range spring and a joint-limit cost fixed it.
A target too close to full reach	Stage B's final target let a straight arm resting on its stop count as holding, a shortcut that could not be unlearned. Feasibility rule 3 now forbids such targets.
Advancing the curriculum too soon	Seeds tightened a rung after one good round and collapsed. It now takes two consecutive good rounds.
Warm-starting from an easier task	Starting three-target training from a two-target model was worse than training from scratch.
Memorising instead of reaching	Every fixed-target model scored 0 % on new targets — which is why R3 trains on random ones.
A reward that punished progress	In the first free-order run, finishing a target made the per-step reward worse, and the policy learned to reach one target and stop.
Retraining that bought nothing	Retraining on the corrected target set (R4) was statistically no better than R3 ($p = 0.67$); the gain came from the rule itself.

9. Reproducing this work

Every training run records its seed, library versions, configuration and reward weights; on the same machine with the same thread count, the same record reproduces bit-identical weights. The main commands:

command	what it does
python run_campaign.py --stage R3	train five seeds of a stage and pick the best
python scripts/verify_human_limits.py <package>	audit hard limits and human-likeness
python scripts/test_click_points.py <package> --anywhere	test on freely clicked points
python -m rl_armMotion.two_d.gui.app --package <package>	the interactive demo
python scripts/build_report_pdf.py	rebuild this document from the data

References

Fischer, F., Bachinski, M., Klar, M., Fleig, A., & Müller, J. (2021). Reinforcement learning control of a biomechanical model of the upper extremity. *Scientific Reports*, 11, 14445.

Flash, T., & Hogan, N. (1985). The coordination of arm movements: an experimentally confirmed mathematical model. *Journal of Neuroscience*, 5(7), 1688–1703.

Haarnoja, T., Zhou, A., Abbeel, P., & Levine, S. (2018). Soft Actor-Critic: off-policy maximum entropy deep reinforcement learning with a stochastic actor. *ICML*.

Haarnoja, T., et al. (2018). Soft Actor-Critic algorithms and applications. arXiv:1812.05905.

Hill, A. V. (1938). The heat of shortening and the dynamic constants of muscle. *Proceedings of the Royal Society B*, 126, 136–195.

Hogan, N., & Sternad, D. (2009). Sensitivity of smoothness measures to movement duration, amplitude, and arrests. *Journal of Motor Behavior*, 41(6), 529–534.

Morasso, P. (1981). Spatial control of arm movements. *Experimental Brain Research*, 42, 223–227.

Ng, A. Y., Harada, D., & Russell, S. (1999). Policy invariance under reward transformations: theory and application to reward shaping. *ICML*.

Raffin, A., Hill, A., Gleave, A., Kanervisto, A., Ernestus, M., & Dormann, N. (2021). Stable-Baselines3: reliable reinforcement learning implementations. *JMLR*, 22(268), 1–8.

Thelen, D. G. (2003). Adjustment of muscle mechanics model parameters to simulate dynamic contractions in older adults. *Journal of Biomechanical Engineering*, 125(1), 70–77.

Winter, D. A. (2009). *Biomechanics and Motor Control of Human Movement* (4th ed.). Wiley.

Zajac, F. E. (1989). Muscle and tendon: properties, models, scaling, and application to biomechanics and motor control. *Critical Reviews in Biomedical Engineering*, 17(4), 359–411.